Assignment 2: Autonomous Normalization & CRUD Engine Hybrid Database Framework

1. Project Objective

In Assignment 1, the system dynamically classified incoming JSON fields and routed them to appropriate storage backends (SQL or MongoDB). Instead of relying on predefined schemas, the system must analyze the JSON structure and automatically determine: a) How relational tables should be split and linked? b) Whether nested JSON should be embedded or separated into collections? c) How read/write queries should be dynamically generated using metadata. The goal of this assignment is to extend that system by implementing:

- An **automated normalization engine** that determines how relational tables should be structured.
- A **document decomposition strategy** for efficient MongoDB storage.
- A **metadata-driven query engine** capable of generating CRUD operations automatically.

Core Technical Pipeline

- **Phase 1: Schema Registration** User defines the JSON schema for incoming records.
- **Phase 2: Data Ingestion** JSON records are received through the ingestion API.
- **Phase 3: Metadata Interpretation** The system analyzes the schema and stores metadata regarding fields, nesting levels, and data types.
- **Phase 4: Data Classification:** SQL, MongoDB, Buffer
- **Phase 5: Storage Strategy Generation**
  - SQL normalisation engine determines table structure.
  - MongoDB document analyser determines collection strategy.
- **Phase 6: Query Generation** Automatically generate queries for CRUD operations.

Deadline

6:00 PM, 22 March 2026

Instructor: Dr. Yogesh K. Meena
March 7, 2026 • Semester II (2025 - 2026) • CS 432 – Databases(Course Project/Assignment 2)
© 2026 Indian Institute of Technology, Gandhinagar. All rights reserved.

2. System Architecture

The following architecture describes the hybrid storage framework that students must implement.
NOTE:

- Framework users can define a JSON schema to make it easier to define database constraints, such as whether a field is unique, not_null, etc.
- All data sent by the user to the framework should be in JSON format
- All data returned by the framework should be in JSON format.
- You are free to define some constraints for your implementation, like what data or flags should be in JSON, at runtime, what flags should be shared along with data, etc.

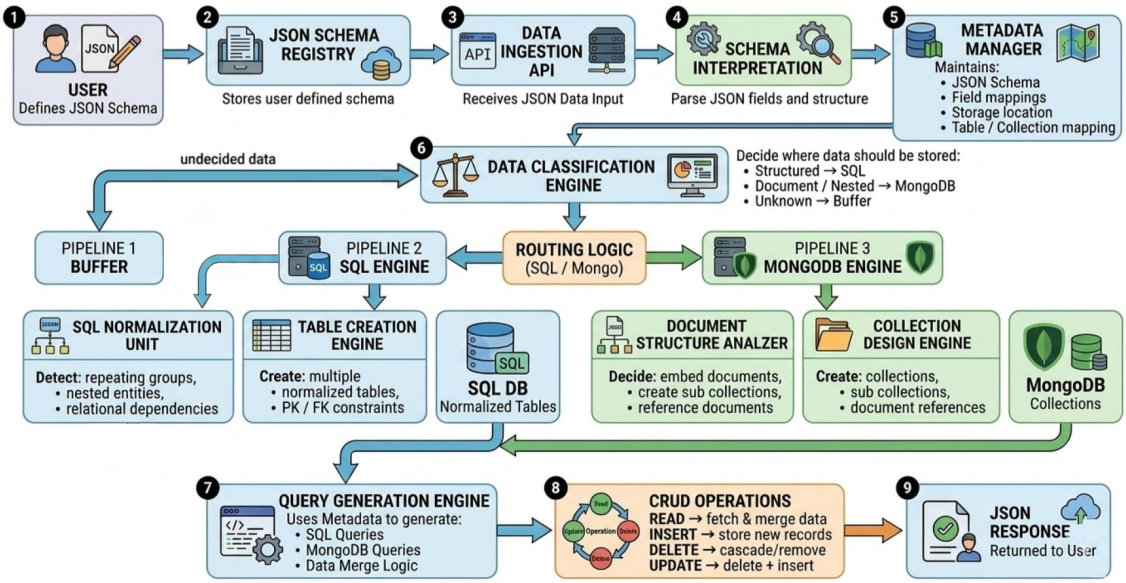

Figure 1: Hybrid ingestion and storage framework

The system consists of three pipelines:

- **Pipeline 1 (Buffer)** Stores undecided fields temporarily until enough information is available to determine placement.
- **Pipeline 2 (SQL Engine)** Responsible for:
  - Detecting repeating entities
  - Creating normalized tables
  - Enforcing referential constraints
- **Pipeline 3 (MongoDB Engine)** Responsible for:
  - Determining document structure
  - Deciding embedding vs referencing
  - Creating collections and sub-collections

All routing decisions and storage mappings must be preserved in a **metadata manager**.

3. SQL Normalization Strategy

Students must design an automated strategy to detect when data should be split into multiple relational tables.

Examples of signals for normalization include:

- Repeating groups
- Nested JSON arrays
- Functional dependencies
- One-to-many relationships

For example:

```
{
  "username": "user1",
  "post_id": 123,
  "comments": [
     {"text":"nice", "time":123},
     {"text":"great", "time":124}
  ]
}
```

Possible relational decomposition:

- USERS table
- POSTS table
- COMMENTS table

Tables must include appropriate:

- Primary keys
- Foreign keys
- Indexes

4. MongoDB Document Strategy

Students must define a method to decide when data should be:

- Embedded inside a document
- Stored in a separate collection

Example considerations:

- **Embedding** Suitable when nested data is small and rarely updated.
- **Referencing** Preferred when:
  - documents grow too large
  - nested elements update frequently
  - data is shared across documents

The system must automatically generate collection schemas based on these decisions.

5. Metadata Driven Query Engine

Once the storage structure is created, the system must allow users to perform CRUD operations through a simple JSON interface.

Example user query:

```
{
  "operation": "read",
  "fields": ["username","comments"]
}
```

Using metadata, the system should:

- Determine where each field is stored
- Generate SQL queries
- Generate MongoDB queries
- Merge results into a final JSON response

6. CRUD Operations

Students must design automated strategies for:

Read

- Translate requested fields into SQL/Mongo queries
- Perform joins or document lookups
- Merge results

Insert

- Split JSON record based on storage mapping
- Insert into SQL tables and MongoDB collections
- Maintain join keys and metadata consistency

Delete

- Delete entire user record (System must handle cascading deletion across SQL and MongoDB)
- Delete specific entity (example: comment)

Update

To ensure schema consistency across backend updates, we can either delete existing records and add new ones or use an alternative approach.

7. Mandatory Report Questions

Teams must submit a technical report describing:

1. **Normalization Strategy** How did your system automatically detect repeating entities and generate normalised SQL tables?
2. **Table Creation Logic** What rules were used to decide primary keys and foreign keys?
3. **MongoDB Design Strategy** How does your system decide between embedded documents and separate collections?
4. **Metadata System** What information is stored in metadata and how is it used to generate queries?
5. **CRUD Query Generation** How does your system translate a user JSON request into SQL and MongoDB queries?
6. **Performance Considerations** How does your design reduce query complexity or document rewriting?
7. **Sources of Information** What documentation, research papers, or books helped guide your implementation?

8. Marking Criteria

Criterion	Focus Area
Normalization Strategy	Quality of relational decomposition
MongoDB Design	Logical document and collection strategy
Query Engine	Ability to generate SQL/Mongo queries automatically
Metadata System	Accuracy of routing and schema tracking
CRUD Functionality	Correct implementation of read, insert, update, delete
Report Quality	Explanation of design decisions and references

9. Conclusion

This assignment focuses on building the **intelligence layer** of the hybrid storage engine. Students must design algorithms that automatically determine database structure and generate queries without manual schema definitions.

The final system should demonstrate how metadata-driven architectures can enable flexible, scalable data storage across both relational and document databases.

© 2026 Indian Institute of Technology, Gandhinagar. All rights reserved.